

Docker

Containerization in Modern Software Delivery

1. Introduction

Docker is an open-source platform that packages an application together with its libraries, binaries, configuration files and runtime dependencies into a single, portable unit called a container. Released in 2013, Docker did not invent container technology, but it made it accessible: it wrapped existing Linux kernel features in a simple command-line interface, a declarative build file and a public registry of ready-made images. Within a few years it had become a default building block of modern software delivery.

The problem Docker solves is familiar to every development team. Software that runs correctly on a developer's laptop frequently fails on a colleague's machine, on the test server, or in production, because each environment carries a slightly different operating system, library version or configuration value. Docker removes this class of failure by shipping the environment along with the application. The container that is tested is the exact artifact that is deployed.

This report explains what Docker is and how it differs from virtual machines, describes its architecture and the Dockerfile format, reviews the commands and tools used in practice, and closes with an assessment of its benefits, limitations and security implications.

2. Containers versus Virtual Machines

Both approaches provide isolation, but at different layers of the stack. A virtual machine virtualizes hardware: a hypervisor presents virtual CPUs, memory and disks, and each machine runs a complete guest operating system with its own kernel. A container virtualizes the operating system instead. Containers on a host share the host kernel and are separated by kernel primitives: namespaces, which give each container its own view of processes, network interfaces and mount points, and control groups (cgroups), which limit the CPU, memory and I/O it may consume.

Because a container carries no guest kernel it is dramatically lighter: images are measured in megabytes rather than gigabytes, and startup in milliseconds rather than the tens of seconds a virtual machine needs to boot. The trade-off is the isolation boundary, since a hypervisor is a harder security barrier than a shared kernel, which is why containers are often run inside virtual machines in multi-tenant settings.

Aspect	Virtual Machine	Container
Isolation layer	Hardware, via a hypervisor	Operating system, via kernel namespaces
Kernel	Each VM runs its own guest kernel	All containers share the host kernel
Typical size	Several gigabytes	Tens to hundreds of megabytes
Start-up time	Tens of seconds to minutes	Milliseconds to a few seconds
Density per host	Tens of instances	Hundreds of instances
Resource overhead	Full guest OS per instance	Only the application process tree
Isolation strength	Stronger, hardware-level boundary	Weaker, shared-kernel boundary

Table 1. Comparison of virtual machines and containers.

The two technologies are complementary rather than competing. A typical cloud deployment runs virtual machines as the unit of infrastructure and hardware isolation, and packs many containers inside each of them as the unit of application packaging and scheduling, combining a strong security boundary with the efficiency and speed of containers.

3. Core Architecture and Concepts

Docker follows a client-server design. The Docker client is the command-line tool developers type into; it sends requests over a REST API to the Docker daemon (dockerd), a background process that builds images, starts containers, manages networks and stores volumes. Because the two communicate over an API, the client can equally control a daemon on a remote host. Below the daemon sit the containerd runtime and runc, which creates the isolated process according to the Open Container Initiative (OCI) specification.

3.1 Images

An image is a read-only template: a filesystem snapshot plus metadata describing the command to run, environment variables and exposed ports. Images are built in layers, and each layer stores only the differences introduced by one build instruction. Layers are content-addressed and cached, so unchanged layers are reused across rebuilds and shared between images, which saves both build time and disk space.

3.2 Containers

A container is a running instance of an image, created by adding a thin writable layer on top of the image's read-only layers. Any file the process modifies is copied into that writable layer, leaving the image untouched. Many containers can therefore run from one image simultaneously, each with its own independent state, and deleting a container discards only its writable layer.

3.3 Registries

A registry distributes images. Docker Hub is the best-known public registry, while organisations run private registries or use cloud offerings such as Amazon ECR, Google Artifact Registry or GitHub Container Registry. The push and pull commands move tagged images between a local host and a registry.

3.4 Volumes and Networks

Containers are ephemeral by design, so any data that must survive a container's life is written to a volume, a storage area managed by Docker outside the container filesystem. Networking is handled by drivers: the default bridge network connects containers on one host and lets them resolve each other by name, host networking removes isolation for performance, and overlay networks span multiple hosts in a cluster.

4. The Dockerfile

A Dockerfile is a plain-text script of instructions that defines how an image is built, making the build reproducible and reviewable in version control. The example below uses a multi-stage build: the first stage compiles the application with the full toolchain, and the second copies only the finished artifact into a minimal runtime image, so build dependencies never reach production.

```
# ---- Stage 1: build ----
FROM node:20 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# ---- Stage 2: runtime ----
FROM node:20-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
USER node
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

Figure 1. A multi-stage Dockerfile for a Node.js service.

5. Essential Commands

Day-to-day work with Docker revolves around a small set of commands covering the lifecycle of images and containers.

Command	Purpose
<code>docker build -t app:1.0 .</code>	Build an image from the Dockerfile in the current directory.
<code>docker run -d -p 8080:3000 app:1.0</code>	Start a container in the background and map a host port.
<code>docker ps -a</code>	List running containers, or all containers with <code>-a</code> .
<code>docker logs -f <container></code>	Stream the standard output of a container.
<code>docker exec -it <container> sh</code>	Open an interactive shell inside a running container.
<code>docker stop / docker rm</code>	Stop a container, then remove it.
<code>docker images / docker rmi</code>	List local images, or delete one.
<code>docker push / docker pull</code>	Upload an image to a registry, or download one.

Table 2. Frequently used Docker commands.

6. Docker Compose

Real applications rarely consist of a single container. Docker Compose describes a multi-container application in one YAML file, defining each service, its image or build context, ports, environment variables, volumes and dependencies. One command then brings the whole stack up or tears it down, which makes Compose the standard way to reproduce a complete development environment.

```
services:
  api:
    build: .
    ports: ["8080:3000"]
    environment:
      DATABASE_URL: postgres://app:secret@db:5432/app
    depends_on: [db]
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: secret
    volumes: ["pgdata:/var/lib/postgresql/data"]

volumes:
  pgdata:
```

Figure 2. A Compose file describing an API and its database.

7. Benefits in Practice

- **Consistency.** The same image runs identically on a laptop, in continuous integration and in production, removing environment drift.
- **Speed and density.** Containers start in milliseconds and share the host kernel, so a single machine can host far more workloads than with virtual machines.
- **Isolation of dependencies.** Two applications requiring conflicting versions of the same library can coexist on one host without interference.
- **Microservices enablement.** Independent services are built, versioned, scaled and replaced separately, each in its own container.
- **Simpler CI/CD.** Pipelines build an immutable image once and promote that exact artifact through every stage, making rollbacks a matter of running a previous tag.
- **Portability.** Because images follow the OCI standard, they run on any compliant runtime and on any major cloud provider.

8. Limitations and Security Considerations

Docker is not a universal solution, and understanding its boundaries is part of using it well. Because containers share the host kernel, a kernel vulnerability can in principle affect every container on the machine; workloads with strict isolation requirements are therefore often placed inside virtual machines or run on sandboxed runtimes. Containers are also tied to the host kernel's operating system family, so Linux images require a Linux kernel, which is supplied by a lightweight virtual machine on Windows and macOS.

Stateful workloads such as databases need careful volume and backup design, since the container filesystem is disposable, and graphical or hardware-bound applications remain awkward to containerize. Operating containers at scale also introduces real complexity in orchestration, monitoring, logging and networking.

8.1 Security Practices

- Start from minimal, official base images such as Alpine or distroless variants to reduce the attack surface and the number of packages to patch.
- Never run the process as root; create an unprivileged user and switch to it with the USER instruction.
- Pin image versions by explicit tag or digest instead of relying on the moving latest tag, so builds remain reproducible.
- Keep secrets out of images. Passwords and keys baked into layers stay recoverable; use runtime secret management instead.
- Scan images continuously for known vulnerabilities and rebuild regularly so that upstream security fixes are actually applied.
- Apply the principle of least privilege: drop unnecessary Linux capabilities, avoid privileged mode, and mount filesystems read-only where possible.

9. The Wider Ecosystem

Docker is one component of a larger ecosystem. The Open Container Initiative standardises the image and runtime formats, so Docker images run equally on engines such as containerd, Podman or CRI-O. In production, orchestration is normally handled by Kubernetes, which schedules containers across many machines and provides self-healing, service discovery, rolling updates and automatic scaling; Docker's own Swarm is simpler but far less widely adopted. Build tools such as BuildKit, Buildah and Kaniko offer faster or daemonless image construction, while services like AWS Fargate and Google Cloud Run execute containers without managing servers at all.

10. Conclusion

Docker changed how software is packaged and shipped by turning an application and its entire runtime environment into a single immutable, portable artifact. Its layered images, declarative Dockerfiles and registry-based distribution made containers practical for ordinary development teams, and in doing so it underpinned the shift toward microservices, continuous delivery and cloud-native infrastructure.

Its advantages, consistency across environments, efficient use of resources, fast start-up and clean dependency isolation, are balanced by a weaker isolation boundary than virtualization, care required for stateful data, and real operational complexity at scale. Used deliberately, with minimal base images, unprivileged users, pinned versions and continuous scanning, Docker remains one of the most valuable tools in modern software engineering, and the foundation on which orchestration platforms such as Kubernetes are built.

End of report